

Garage Conveyor

Live Monitoring Quick Start

Get live PLC data on your screen in 5 minutes

Everything is already built. This guide walks you through turning it on.

PLC	Micro820 at 192.168.1.100 (LAN)
VFD	GS10 via RS-485 Modbus RTU
CHARLIE	192.168.1.12 (same LAN)
Dashboard	Node-RED at http://192.168.1.12:1880/dashboard
Date	May 27, 2026

Architecture (What's Already Built)

```
Micro820 PLC (192.168.1.100:502)
| Modbus TCP @ 1 Hz
v
tools/demo_plc_poller.py (514 lines, async, READ-ONLY)
| HTTP POST /ingest
v
mira-relay/relay_server.py (228 lines, Starlette)
| Writes equipment_status + faults to SQLite
v
mira-bridge/data/mira.db (SQLite, shared volume)
| Read every 10s by Node-RED
v
Node-RED Dashboard 2.0
| Gauges + Fault Log + Ask MIRA chat
v
http://192.168.1.12:1880/dashboard
```

TIP: There is also a standalone simulator (tools/demo_plc_simulator.py) that emulates the Micro820 on port 5020. You can test the entire chain without the PLC powered up.

TIER 1: Terminal Dashboard (5 minutes)

Zero infrastructure. Just a Python script that polls the PLC and shows live data in your terminal. Start here to prove connectivity.

Step 1: Open a terminal on CHARLIE

Step 2: Verify PLC connectivity

```
cd ~/MIRA
python plc/test_modbus.py 192.168.1.100
```

Should print all coils + holding registers. If it fails, check the PLC is powered and on the network.

Step 3: Start the live monitor

```
python plc/live_monitor.py --host 192.168.1.100 --poll 0.5
```

What you'll see:

Data	Source Register	Update Rate
Motor Running / Stopped	Coil 0-1	0.5s
Motor Speed (0-100)	Register 100	0.5s
Motor Current (amps)	Register 101 / 10	0.5s
Temperature	Register 102 / 10	0.5s
Conveyor Speed	Register 104	0.5s
Fault/Error Code	Register 105	0.5s
E-Stop Status	Coil 6	0.5s
Sensor 1 / Sensor 2	Coils 4-5	0.5s

WARNING: The keyboard controls (F=fwd, R=rev, S=stop) use msvcrt which is Windows-only. On macOS, the display works but keyboard shortcuts won't. Drive the VFD from the panel or CCW instead.

TIER 2: Web Dashboard (30 minutes)

Full browser-based dashboard with gauges, fault log, and MIRA chat. Viewable from any device on the LAN.

Step 1: Start Node-RED + mira-relay

```
cd ~/MIRA
docker compose up -d node-red mira-relay
sleep 5
curl -f http://localhost:1880/ # Node-RED OK?
curl -f http://localhost:8765/health # Relay OK?
```

Step 2: Import the conveyor dashboard flow

Open <http://localhost:1880> in your browser.

Click the hamburger menu (top right) > Import.

Upload: **mira-bridge/flows/mira-dashboard-conveyor.json**

Click **Deploy** (top right, red button).

Step 3: Bench-test with the simulator (no PLC needed)

```
# One command does everything:
bash tools/bench-test-conveyor.sh

# Or manually:
python -m tools.demo_plc_simulator --port 5020 &
python -m tools.demo_plc_poller \
--plc-ip 127.0.0.1 --plc-port 5020 \
--relay-url http://localhost:8765 \
--poll-interval 1.0
```

Open <http://localhost:1880/dashboard> — gauges should animate from the simulator's ramps.

Step 4: Switch to the real PLC

```
# Stop the simulator (Ctrl+C or kill %1)
# Then:
bash tools/run-conveyor-live.sh 192.168.1.100

# Or manually:
python -m tools.demo_plc_poller \
--plc-ip 192.168.1.100 \
--relay-url http://localhost:8765 \
--poll-interval 1.0
```

TIP: The dashboard URL works from any device on your LAN. Open <http://192.168.1.12:1880/dashboard> on your phone to monitor the conveyor from anywhere in the garage.

TIER 3: NeonDB Historian (Optional, +1 hour)

Adds time-series storage to NeonDB so MIRA can answer questions like 'what was the motor temperature an hour ago?' or 'when did the last fault clear?'

```
doppler run -p factorylm -c prd -- python -m tools.demo_plc_poller \  
--plc-ip 192.168.1.100 \  
--relay-url http://localhost:8765 \  
--db-url "$NEON_DATABASE_URL" \  
--poll-interval 1.0
```

The poller auto-creates live_signal_cache + live_signal_events tables on first write. Engine-side query tools for these tables are a follow-up (not wired into mira-mcp yet).

Troubleshooting

Problem	Fix
test_modbus.py: Connection refused	PLC not powered, wrong IP, or not on LAN. Check with: ping 192.168.1.100
test_modbus.py: Timeout	PLC is on but Modbus TCP not enabled. Verify in CCW: Serial Port > Modbus RTU Master enabled
Node-RED won't start	Port 1880 in use? Check: docker ps. Kill conflicting container.
Relay health returns error	Check docker logs mira-relay. Usually a missing volume mount.
Dashboard shows no data	equipment_id mismatch — poller must write CONV-001, not conveyor001. Fixed in latest code.
Simulator works but real PLC doesn't	Firewall on CHARLIE blocking port 502? Try: nc -zv 192.168.1.100 502
Gauges stuck / not updating	Check poller is running (should print a line per poll). Check relay health. Click Deploy in Node-RED.

File Inventory

File	Lines	What It Does
plc/test_modbus.py	154	One-shot connectivity test — reads all coils + registers
plc/live_monitor.py	416	Rich terminal dashboard with live data + keyboard controls
tools/demo_plc_poller.py	514	Async Modbus poll > relay POST + optional NeonDB
tools/demo_plc_simulator.py	293	Fake Micro820 on port 5020 with ramps + fault triggers
tools/bench-test-conveyor.sh	~30	One-shot: starts Node-RED + relay + simulator + poller
tools/run-conveyor-live.sh	~20	One-shot: starts Node-RED + relay + real PLC poller
mira-relay/relay_server.py	228	Starlette /ingest + /ws + /health, writes SQLite
mira-bridge/flows/mira-dashboard-conveyor.json	-	Node-RED Dashboard 2.0 flow (import into Node-RED)
docker-compose.override.yml	~15	Adds mira-relay to local Docker Compose stack

Quick Reference Commands

```
# Test PLC connectivity
python plc/test_modbus.py 192.168.1.100

# Terminal dashboard (fastest)
python plc/live_monitor.py --host 192.168.1.100

# Bench test with simulator (no PLC)
bash tools/bench-test-conveyor.sh

# Live dashboard with real PLC
bash tools/run-conveyor-live.sh 192.168.1.100

# Dashboard URL (any device on LAN)
http://192.168.1.12:1880/dashboard
```